РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

дисциплина: Архитектура компьютера

Студент: Тимофеева
Милана Александровна

Группа: НБИбд-02-25

МОСКВА

2025 г.

Оглавление

1. Цель работы.....3
2. Теоретическое введение..... 3
2.1. Основы работы с Midnight Commander.....3
2.2. Структура программы на языке ассемблера NASM.....4
2.3. Элементы программирования на mov.....8
2.4. Элементы программирования на int.....9
2.4. Системные вызовы для обеспечения диалога с пользователем.....9
3. Порядок выполнения лабораторной работы.....10
3.1. Подключение внешнего файла in_out.asm.....16
4. Задание для самостоятельной работы.....23
5. Вывод..... 25

1. Цель работы

-Приобретение практических навыков работы в Midnight Commander. Освоение инструкций языка ассемблера mov и int.

2. Теоретическое введение

2.1. Основы работы с Midnight Commander

-Midnight Commander (или просто mc) — это программа, которая позволяет просматривать структуру каталогов и выполнять основные операции по управлению файловой системой, т.е. mc является файловым менеджером. Midnight Commander позволяет сделать работу с файлами более удобной и наглядной.

-Для активации оболочки Midnight Commander достаточно ввести в командной строке mc и нажать клавишу Enter (рис. 2.1).

-В Midnight Commander используются функциональные клавиши F1 — F10 , к которым привязаны часто выполняемые операции (рис. 2.1)

Функцио-нальные клавиши	Выполняемое действие
F1	вызов контекстно-зависимой подсказки
F2	вызов меню, созданного пользователем
F3	просмотр файла, на который указывает подсветка в активной панели
F4	вызов встроенного редактора для файла, на который указывает подсветка в активной панели

Рис.2.1. Функциональные клавиши Midnight Commander

F5	копирование файла или группы отмеченных файлов из каталога, отображаемого в активной панели, в каталог, отображаемый на второй панели
F6	перенос файла или группы отмеченных файлов из каталога, отображаемого в активной панели, в каталог, отображаемый на второй панели
F7	создание подкаталога в каталоге, отображаемом в активной панели
F8	удаление файла (подкаталога) или группы отмеченных файлов
F9	вызов основного меню программы
F10	выход из программы

Рис.2.1. Функциональные клавиши Midnight Commander

-Следующие комбинации клавиш облегчают работу с Midnight Commander:

- Tab используется для переключениями между панелями;
- ↑ и ↓ используется для навигации, Enter для входа в каталог или открытия файла (если в файле расширений mc.ext заданы правила связи определённых расширений файлов с инструментами их запуска или обработки);
- Ctrl + u (или через меню Команда > Переставить панели) меняет местами содержимое правой и левой панелей;
- Ctrl + o (или через меню Команда > Отключить панели) скрывает или возвращает панели Midnight Commander, за которыми доступен для работы командный интерпретатор оболочки и выводимая туда информация.
- Ctrl + x + d (или через меню Команда > Сравнить каталоги) позволяет сравнить содержимое каталогов, отображаемых на левой и правой панелях.

2.2. Структура программы на языке ассемблера NASM

-Программа на языке ассемблера NASM, как правило, состоит из трёх секций: секция кода программы (SECTION .text), секция инициированных (известных во время компиляции) данных (SECTION .data) и секция неинициализированных данных (тех, под которые во время компиляции только отводится память, а значение присваивается в ходе выполнения программы) (SECTION .bss).

-Таким образом, общая структура программы имеет следующий вид: (рис.2.2.)

```
SECTION  .data    ; Секция содержит переменные, для
   ...            ; которых задано начальное значение
```

Рис.2.2. Вид общей структуры программы

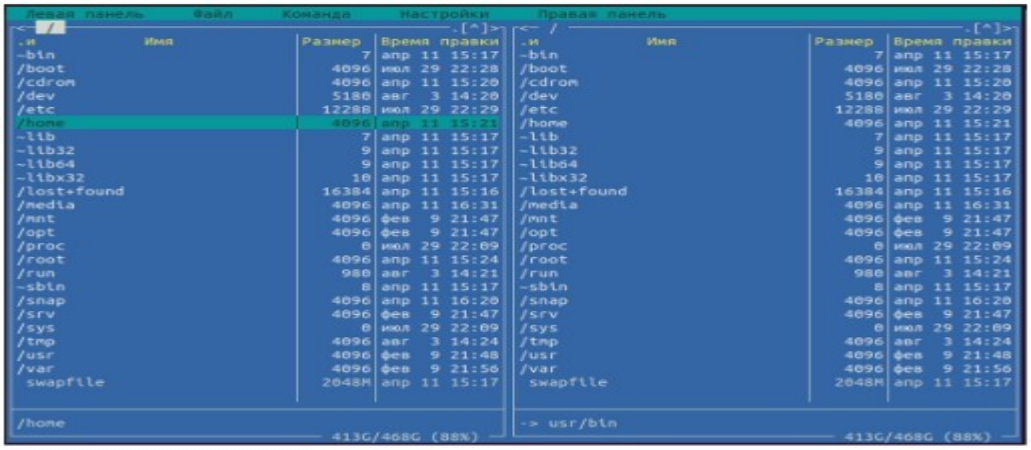


Рис.2.3. Окно Midnight Commander

SECTION .bss ; Секция содержит переменные, для

... ; которых не задано начальное значение

SECTION .text ; Секция содержит код программы

GLOBAL _start

_start: ; Точка входа в программу

... ; Текст программы

mov eax,1 ; Системный вызов для выхода (sys_exit)

mov ebx,0 ; Выход с кодом возврата 0 (без ошибок)

int 80h ; Вызов ядра

-Для объявления инициированных данных в секции .data используются директивы DB, DW, DD, DQ и DT, которые резервируют память и указывают, какие значения должны храниться в этой памяти:

- DB (define byte) — определяет переменную размером в 1 байт;

- DW (define word) — определяет переменную размеров в 2 байта (слово);
- DD (define double word) — определяет переменную размером в 4 байта (двойное слово);
- DQ (define quad word) — определяет переменную размером в 8 байт (учетверённое слово);
- DT (define ten bytes) — определяет переменную размером в 10 байт.

-Директивы используются для объявления простых переменных и для объявления массивов. Для определения строк принято использовать директиву DB в связи с особенностями хранения данных в оперативной памяти.

-Синтаксис директив определения данных следующий: (Рис.2.4.)

<имя> DB <операнд> [, <операнд>] [, <операнд>]

Пример	Пояснение
`a db 10011001b`	определяем переменную a размером 1 байт с начальным значением, заданным в двоичной системе счисления (на двоичную систему счисления указывает также буква b (binary) в конце числа)
`b db '!'`	определяем переменную b в 1 байт, инициализируемую символом !
`c db "Hello"`	определяем строку из 5 байт
`d dd -345d`	определяем переменную d размером 4 байта с начальным значением, заданным в десятичной системе счисления (на десятичную систему указывает буква d (decimal) в конце числа)
`h dd 0f1ah`	определяем переменную h размером 4 байта с начальным значением, заданным в шестнадцатеричной системе счисления (h — hexadecimal)

Рис.2.4. Пример

-Для объявления неинициированных данных в секции .bss используются директивы resb, resw, resd и другие, которые сообщают ассемблеру, что необходимо зарезервировать заданное количество ячеек памяти. Примеры их использования приведены в рис. 2.5.

Директива	Назначение директивы	Аргумент	Назначение аргумента
`resb`	Резервирование заданного числа однобайтовых ячеек	`string resb 20`	По адресу с меткой `string` будет расположен массив из 20 однобайтовых ячеек (хранение строки символов)
`resw`	Резервирование заданного числа двухбайтовых ячеек (слов)	`count resw 256`	По адресу с меткой `count` будет расположен массив из 256 двухбайтовых слов
`resd`	Резервирование заданного числа четырёхбайтовых ячеек (двойных слов)	`x resd 1`	По адресу с меткой `x` будет расположено одно двойное слово (т.е. 4 байта для хранения большого числа)

Рис.2.5. Директивы для объявления неинициированных данных

2.3. Элементы программирования на mov

-Инструкция языка ассемблера mov предназначена для дублирования данных источника в приёмнике. В общем виде эта инструкция записывается в виде

mov dst,src

Здесь операнд dst — приёмник, а src — источник. В качестве операнда могут выступать регистры (register), ячейки памяти (memory) и непосредственные значения (const). В рис. 2.6 приведены варианты использования mov с разными операндами.

Тип операндов	Пример	Пояснение
mov <reg>,<reg>	mov eax,ebx	пересылает значение регистра ebx в регистр eax
mov <reg>,<mem>	mov cx,[eax]	пересылает в регистр cx значение из памяти, указанной в eax
mov <mem>,<reg>	mov rez,ebx	пересылает в переменную rez значение из регистра ebx
mov <reg>,<const>	mov eax,403045h	пишет в регистр eax значение 403045h
mov <mem>,<const>	mov byte[rez],0	записывает в переменную rez значение 0

Рис.2.6. Варианты использования mov с разными операндами

-ВАЖНО! Переслать значение из одной ячейки памяти в другую нельзя, для этого необходимо использовать две инструкции mov:

mov eax, x

mov y, eax

-Также необходимо учитывать то, что размер операндов приемника и источника должны совпадать. Использование слудующих примеров приведет к ошибке:

- mov al,1000h — ошибка, попытка записать 2-байтное число в 1-байтный регистр;
- mov eax,cx — ошибка, размеры операндов не совпадают.

2.4. Элементы программирования на int

-Инструкция языка ассемблера intпредназначена для вызова прерывания с указанным номером. В общем виде она записывается в виде int n

-Здесь n — номер прерывания, принадлежащий диапазону 0–255.

-При программировании в Linux с использованием вызовов ядра sys_calls n=80h (принято задавать в шестнадцатеричной системе счисления).

-После вызова инструкции int 80h выполняется системный вызов какой-либо функции ядра Linux. При этом происходит передача управления ядру операционной системы. Чтобы узнать, какую именно системную функцию нужно выполнить, ядро извлекает номер системного вызова из регистра eax. Поэтому перед вызовом прерывания необходимо поместить в этот регистр нужный номер. Кроме того, многим системным функциям требуется передавать какие-либо параметры. По принятым в ОС Linux правилам эти параметры помещаются в порядке следования в остальные регистры процессора: ebx, ecx, edx. Если системная функция должна вернуть значение, то она помещает его в регистр eax.

2.4. Системные вызовы для обеспечения диалога с пользователем

-Простейший диалог с пользователем требует наличия двух функций — вывода текста на экран и ввода текста с клавиатуры. Простейший способ вывести строку на экран — использовать системный вызов write. Этот системный вызов имеет номер 4, поэтому перед вызовом инструкции int необходимо поместить

значение 4 в регистр eax. Первым аргументом write, помещаемым в регистр ebx, задаётся дескриптор файла. Для вывода на экран в качестве дескриптора файла нужно указать 1 (это означает «стандартный вывод», т. е. вывод на экран). Вторым аргументом задаётся адрес выводимой строки (помещаем его в регистр ecx, например, инструкцией mov ecx, msg). Строка может иметь любую длину. Последним аргументом (т.е. в регистре edx) должна задаваться максимальная длина выводимой строки.

-Для ввода строки с клавиатуры можно использовать аналогичный системный вызов read. Его аргументы – такие же, как у вызова write, только для «чтения» с клавиатуры используется файловый дескриптор 0 (стандартный ввод).

-Системный вызов exit является обязательным в конце любой программы на языке ассемблер. Для обозначения конца программы перед вызовом инструкции int 80h необходимо поместить в регистр eax значение 1, а в регистр ebx код завершения 0.

3. Порядок выполнения лабораторной работы

-1. Открыла Midnight Commander: (рис.3.1)

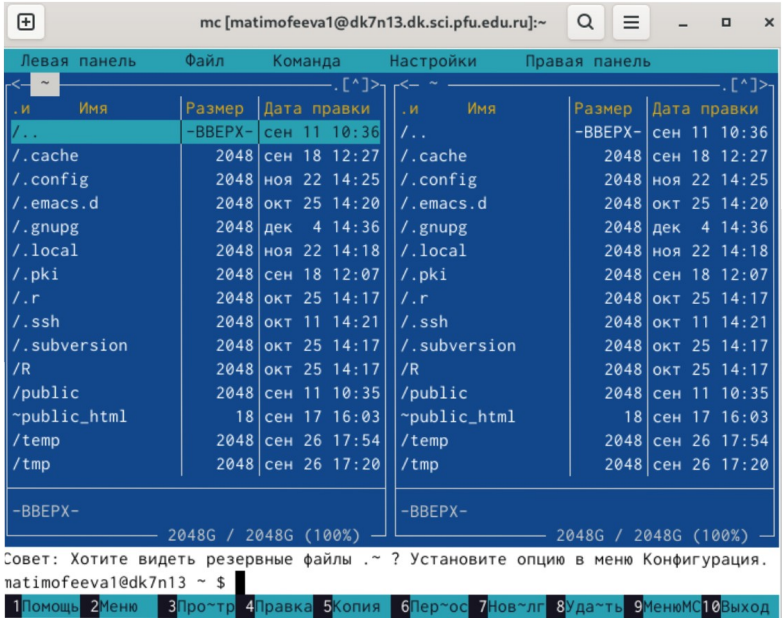


Рис.3.1. Открытие Midnight Commander

-2. Пользуясь клавишами ↑ , ↓ и Enter перешла в каталог ~/work/arch-pc созданный при выполнении лабораторной работы №4.

-3. С помощью функциональной клавиши F7 создала папку lab05 и перешла в созданный каталог. (рис.3.2)

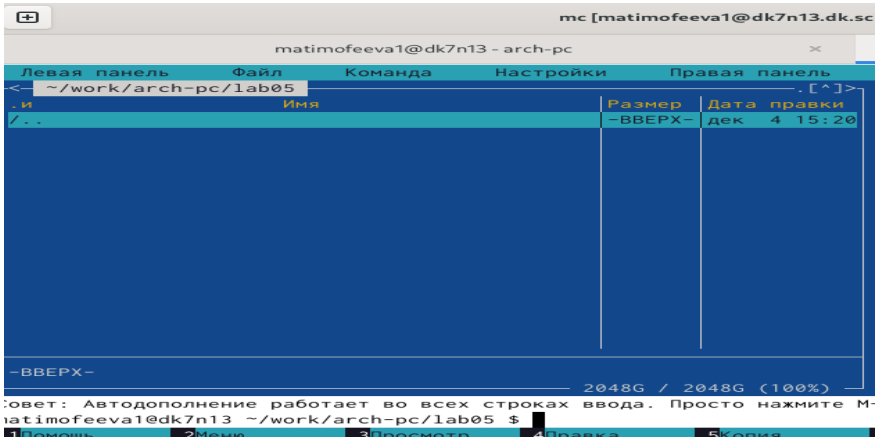


Рис.3.2. Создание папки lab05

-4. Пользуясь строкой ввода и командой touch создала файл lab5-1.asm. (рис.3.3)

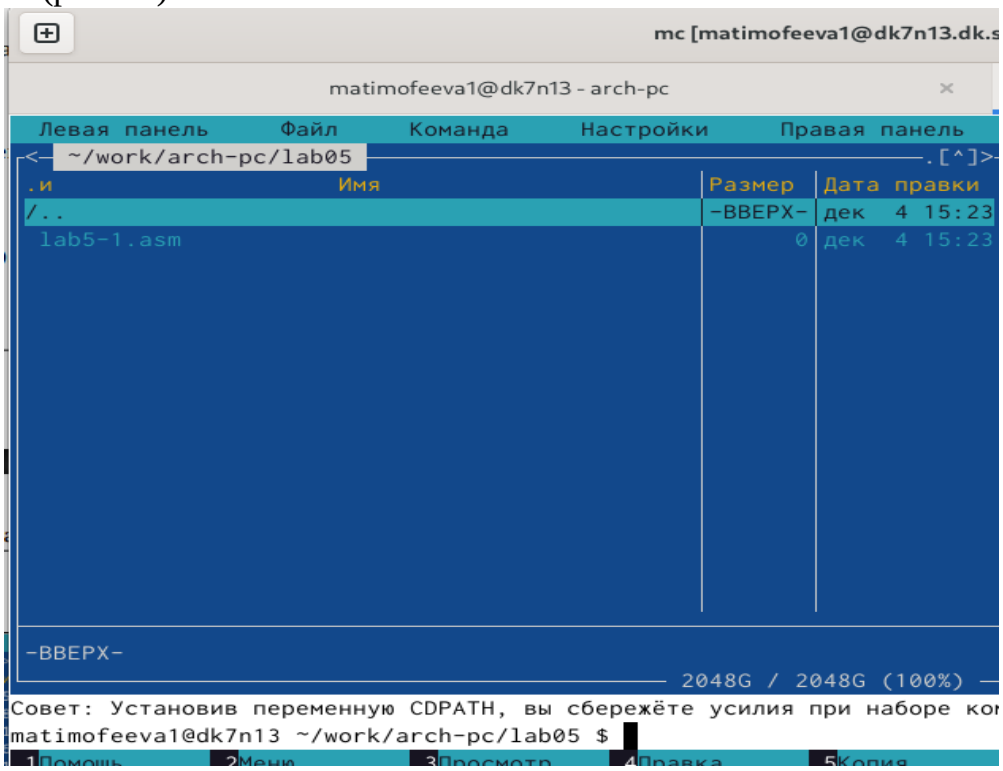


Рис.3.3. Командой touch создание файла lab5-1.asm

-5. С помощью функциональной клавиши F4 открыла файл lab5-1.asm для редактирования во встроенном редакторе. Как правило в качестве встроенного редактора Midnight Commander используется редакторы nano или mcedit. (рис.3.4)

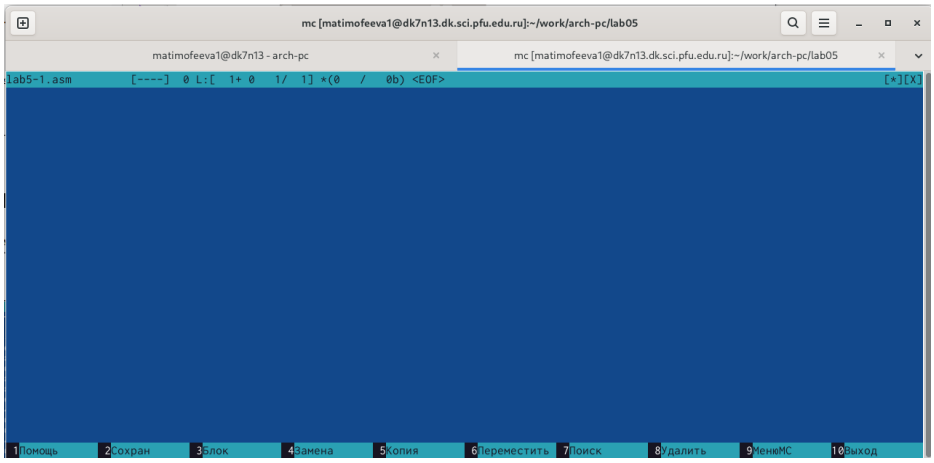


Рис.3.4. Открытие файла lab5-1.asm

-6. Ввела текст программы из листинга 3.5, сохранила изменения и закрыла файл.

```
;-----------------------------------------------------------------
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----------------------------------------------------------------

;-------------------  Объявление переменных  ----------------
  SECTION .data                    ; Секция инициированных данных
  msg:  DB 'Введите строку:',10 ; сообщение плюс
                                   ; символ перевода строки
  msgLen:  EQU $-msg               ; Длина переменной 'msg'

  SECTION  .bss        ; Секция не инициированных данных
  buf1:     RESB 80    ; Буфер размером 80 байт

 ;-------------------     Текст программы    -----------------

    SECTION .text        ; Код программы
    GLOBAL _start        ; Начало программы
     _start:             ; Точка входа в программу

 ;------------ Системный вызов `write`
 ; После вызова инструкции 'int 80h' на экран будет
 ; выведено сообщение  из переменной 'msg' длиной 'msgLen'

     mov  eax,4          ; Системный вызов для записи (sys_write)
     mov  ebx,1          ; Описатель файла 1 - стандартный вывод
     mov  ecx,msg        ; Адрес строки 'msg' в 'ecx'
     mov  edx,msgLen     ; Размер строки 'msg' в 'edx'
     int  80h            ; Вызов ядра

 ;------------  системный вызов `read`  ----------------------
 ; После вызова инструкции 'int 80h' программа будет ожидать ввода
 ; строки, которая будет записана в переменную 'buf1' размером 80 байт

     mov  eax, 3         ; Системный вызов для чтения (sys_read)
     mov  ebx, 0         ; Дескриптор файла 0 - стандартный ввод
     mov  ecx, buf1      ; Адрес буфера под вводимую строку
     mov  edx, 80        ; Длина вводимой строки
     int  80h            ; Вызов ядра

 ;------------ Системный вызов `exit`  ----------------------
 ; После вызова инструкции 'int 80h' программа завершит работу

     mov  eax,1          ; Системный вызов для выхода (sys_exit)
     mov  ebx,0          ; Выход с кодом возврата 0 (без ошибок)
     int  80h            ; Вызов ядра
```

Рис.3.5. Листинг

```
mc [matimofeeva1@dk7n13.dk.sci.pfu.edu.ru]:~/work/arch-pc
lab5-1.asm.        [-M--] 20 L:[  1+ 0   1/ 29] *(24  /2025b) 1080 0x438
SECTION .data ; Секция инициированных данных
msg: DB 'Введите строку:',10 ; сообщение плюс
; символ перевода строки
msgLen: EQU $-msg ; Длина переменной 'msg'
SECTION .bss ; Секция не инициированных данных
buf1: RESB 80 ; Буфер размером 80 байтSECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
;------------ Системный вызов `write`
; После вызова инструкции 'int 80h' на экран будет
; выведено сообщение из переменной 'msg' длиной 'msgLen'
mov eax,4 ; Системный вызов для записи (sys_write)
mov ebx,1 ; Описатель файла 1 - стандартный вывод
mov ecx,msg ; Адрес строки 'msg' в 'ecx'
mov edx,msgLen ; Размер строки 'msg' в 'edx'
int 80h ; Вызов ядра
;------------ системный вызов `read` ----------------------
; После вызова инструкции 'int 80h' программа будет ожидать ввода
; строки, которая будет записана в переменную 'buf1' размером 80 байт
mov eax, 3 ; Системный вызов для чтения (sys_read)
mov ebx, 0 ; Дескриптор файла 0 - стандартный ввод
mov ecx, buf1 ; Адрес буфера под вводимую строку
mov edx, 80 ; Длина вводимой строки
int 80h ; Вызов ядра
;------------ Системный вызов `exit` ----------------------
; После вызова инструкции 'int 80h' программа завершит работу
mov eax,1 ; Системный вызов для выхода (sys_exit)
mov ebx,0 ; Выход с кодом возврата 0 (без ошибок)
int 80h ; Вызов ядра
```

Рис.3.6. Ввод текста программы из листинга 3.5

-7. С помощью функциональной клавиши F3 открыла файл lab5-1.asm для просмотра. Убедилась, что файл содержит текст программы.(рис.3.7.)

```
mc [matimofeeva1@dk7n13.dk.sci.pfu.edu.ru]:~/work/arch-pc
/afs/.dk.sci.pfu.edu.ru/home/m/a/matimofeeva1/work/arch-pc/lab5-1.asm.
SECTION .data ; Секция инициированных данных
msg: DB 'Введите строку:',10 ; сообщение плюс
; символ перевода строки
msgLen: EQU $-msg ; Длина переменной 'msg'
SECTION .bss ; Секция не инициированных данных
buf1: RESB 80 ; Буфер размером 80 байтSECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
;------------ Системный вызов `write`
; После вызова инструкции 'int 80h' на экран будет
; выведено сообщение из переменной 'msg' длиной 'msgLen'
mov eax,4 ; Системный вызов для записи (sys_write)
mov ebx,1 ; Описатель файла 1 - стандартный вывод
mov ecx,msg ; Адрес строки 'msg' в 'ecx'
mov edx,msgLen ; Размер строки 'msg' в 'edx'
int 80h ; Вызов ядра
;------------ системный вызов `read` ----------------------
; После вызова инструкции 'int 80h' программа будет ожидать ввода
; строки, которая будет записана в переменную 'buf1' размером 80 байт
mov eax, 3 ; Системный вызов для чтения (sys_read)
mov ebx, 0 ; Дескриптор файла 0 - стандартный ввод
mov ecx, buf1 ; Адрес буфера под вводимую строку
mov edx, 80 ; Длина вводимой строки
int 80h ; Вызов ядра
;------------ Системный вызов `exit` ----------------------
; После вызова инструкции 'int 80h' программа завершит работу
mov eax,1 ; Системный вызов для выхода (sys_exit)
mov ebx,0 ; Выход с кодом возврата 0 (без ошибок)
int 80h ; Вызов ядра
```

Рис.3.7. Открытие файла lab5-1.asm для просмотра

-8. Оттранслировала текст программы lab5-1.asm в объектный файл. Выполнила компоновку объектного файла и запустила получившийся исполняемый файл. (рис3.9.) Программа выводит строку 'Введите строку:' и ожидает ввода с клавиатуры. На запрос введите Ваши ФИО.(рис.3.8.)

:~/work/arch-pc/lab05$ nasm -f elf lab5-1.asm

:~/work/arch-pc/lab05$ ld -m elf_i386 -o lab5-1 lab5-1.o

:~/work/arch-pc/lab05$ ./lab5-1

Введите строчку:

Тимофеева Милана Александровна

Рис.3.9. Оттранслирование текста программы lab5-1.asm в объектный файл.

3.1. Подключение внешнего файла in_out.asm

-Для упрощения написания программ часто встречающиеся одинаковые участки кода (такие как, например, вывод строки на экран или выход их программы) можно оформить в виде подпрограмм и сохранить в отдельные файлы, а во всех нужных местах поставить вызов нужной подпрограммы. Это позволяет сделать основную программу более удобной для написания и чтения.

-NASM позволяет подключать внешние файлы с помощью директивы %include, которая предписывает ассемблеру заменить эту директиву содержимым файла. Подключаемые файлы также написаны на языке ассемблера. Важно отметить, что директива %include в тексте программы должна стоять раньше, чем встречаются вызовы подпрограмм из подключаемого файла. Для вызова подпрограммы из внешнего файла используется инструкция call,которая имеет следующий вид

call <function>

где function имя подпрограммы.

Для выполнения лабораторных работ использовал файл in_out.asm1:

- slen – вычисление длины строки (используется в подпрограммах печати сообщения для определения количества выводимых байтов);
- sprint – вывод сообщения на экран, перед вызовом sprint в регистр eax необходимо записать выводимое сообщение (mov eax,<message>);
- sprintLF – работает аналогично sprint, но при выводе на экран добавляет к сообщению символ перевода строки;
- sread – ввод сообщения с клавиатуры, перед вызовом sread в регистр eax необходимо записать адрес переменной в которую введенное сообщение буд записано (moveax,<buffer>) , в регистр ebx – длину вводимой строки (mov ebx,<N>);
- iprint – вывод на экран чисел в формате ASCII, перед вызовом iprint в регистр eax необходимо записать выводимое число (mov eax,<int>);

- iprintLF – работает аналогично iprint, но при выводе на экран после числа добавляет к символ перевода строки;
- atoi – функция преобразует ascii-код символа в целое число и записыает результат в регистр eax, перед вызовом atoi в регистр eax необходимо записать число (moveax,<int>);
- quit – завершение программы.

-9. Скачала файл in_out.asm со страницы курса в ТУИС.

-10. Подключаемый файл in_out.asm положила в тот же каталог, что и файл с программой, в которой он используется.(рис.3.10.)

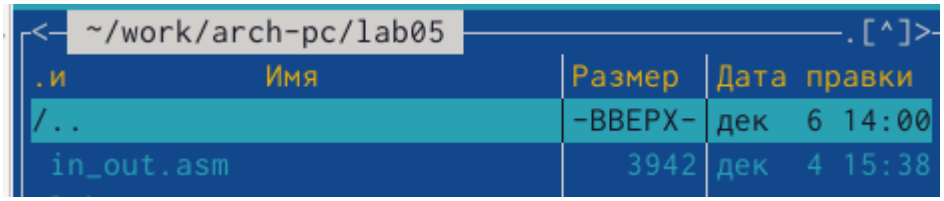


Рис.3.10. Подключение файла in_out.asm

-В одной из панелей mc открыла каталог с файлом lab5-1.asm. В другой панели каталог со скаченным файлом in_out.asm (для перемещения между панелями использовал Tab). Скопировала файл in_out.asm в каталог с файлом lab5-1.asm с помощью функциональной клавиши F5. (рис.3.11)

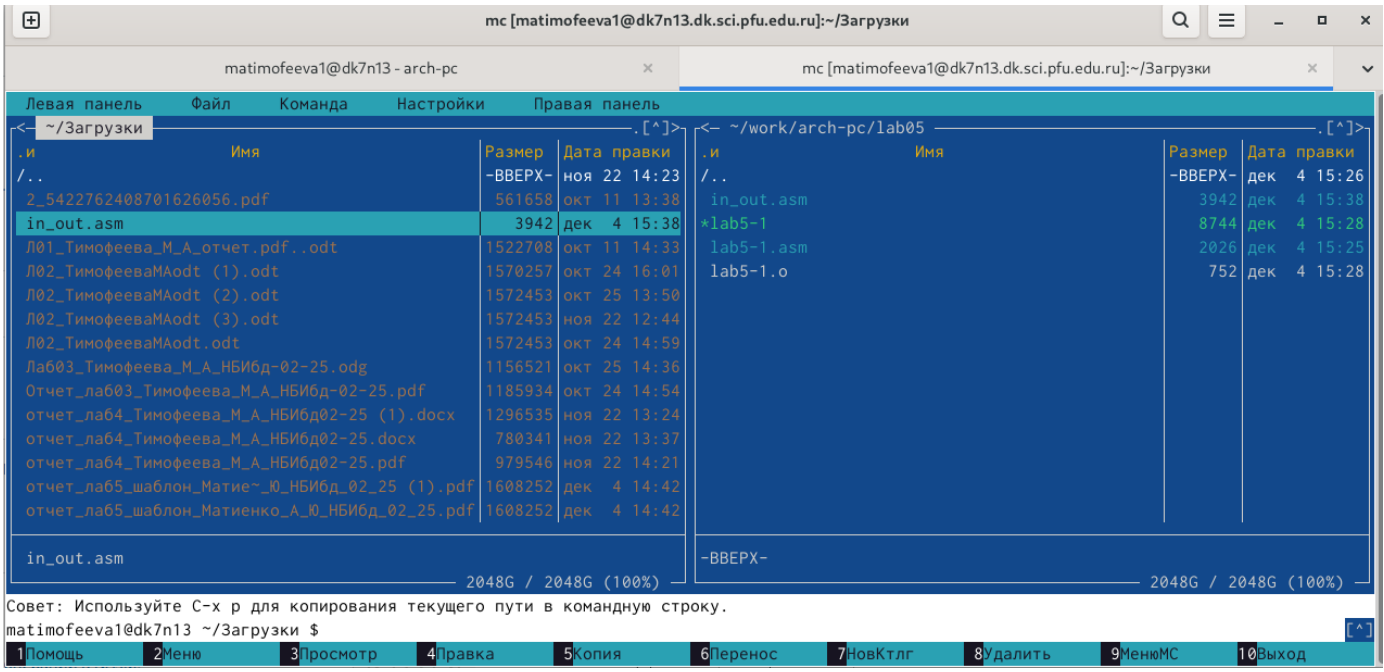


Рис.3.11. Копирование файла in_out.asm в каталог с файлом lab5-1.asm

-11. С помощью функциональной клавиши F6 создала копию файла lab5-1.asm с именем lab5-2.asm. Ввела файл lab5-1.asm, нажала клавишу F6 , ввела имя файла lab5-2.asm и нажала клавишу Enter. (рис.3.12.)

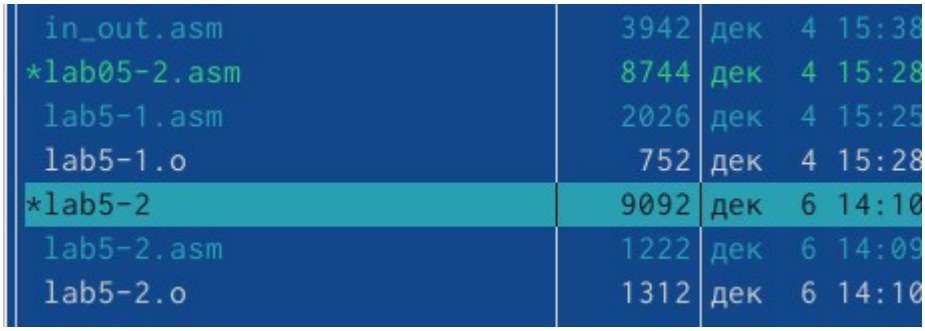


Рис.3.12. Создание копии файла lab5-1.asm с именем lab5-2.asm

-12. Исправила текст программы в файле lab5-2.asm с использование подпрограмм из внешнего файла in_out.asm (рис.3.14.) в соответствии с листингом рис (3.13.) Создала исполняемый файл и проверила его работу. (рис.3.15.)

```
;-------------------------------------------------------------------
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-------------------------------------------------------------------

%include 'in_out.asm'              ; подключение внешнего файла

SECTION .data                    ; Секция инициированных данных
msg:  DB 'Введите строку: ',0h   ; сообщение

SECTION  .bss        ; Секция не инициированных данных
buf1:  RESB 80       ; Буфер размером 80 байт

SECTION .text        ; Код программы
   GLOBAL _start     ; Начало программы
   _start:           ; Точка входа в программу

   mov   eax, msg    ; запись адреса выводимого сообщения в `EAX`
   call  sprintLF    ; вызов подпрограммы печати сообщения

   mov   ecx, buf1   ; запись адреса переменной в `EAX`
   mov   edx, 80     ; запись длины вводимого сообщения в `EBX`


    call  sread       ; вызов подпрограммы ввода сообщения

    call  quit        ; вызов подпрограммы завершения
```

Рис.3.13. Листинг

```
 GNU nano 8.6   /afs/.dk.sci.pfu.edu.ru/home/m/a/matimofeeva1/work/arch-pc/lab05/lab5-2.asm
;-----------------------------------------------------------------
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----------------------------------------------------------------
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data ; Секция инициированных данных
msg: DB 'Введите строку: ',0h ; сообщение
SECTION .bss ; Секция не инициированных данных
buf1: RESB 80 ; Буфер размером 80 байт
SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
mov eax, msg ; запись адреса выводимого сообщения в `EAX`
call sprintLF ; вызов подпрограммы печати сообщения
mov ecx, buf1 ; запись адреса переменной в `EAX`
mov edx, 80 ; запись длины вводимого сообщения в `EBX`
call sread ; вызов подпрограммы ввода сообщения
call quit ; вызов подпрограммы завершения
```

Рис.3.14. Исправление текста программы в файле lab5-2.asm с использование подпрограмм из внешнего файла in_out.asm

```
matimofeeva1@dk7n08 ~ $ cd ~/work/arch-pc/lab05
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ nasm -f elf lab5-2.asm
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ lf -m elf_i386 -o lab5-2 lab5-2.o
bash: lf: команда не найдена
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ ld -m elf_i386 -o lab5-2 lab5-2.o
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ ./lab5-2
Введите строку:
Тимофеева Милана
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $
```

Рис.3.15. Проверка работы файла

-13. В файле lab5-2.asm заменила подпрограмму sprintLF на sprint. Создайте исполняемый файл и проверила его работу.(рис.3.17.), (рис.3.16.)

-Разница в том, что sprintLF перемещает курсор на следующую строку, а sprint оставляет курсор на той же строке.

```
lab5-2.asm        [-M--] 11 L:[  1+12  13/ 17] *(847 /1222b) 0032 0x020                    [*][X]
;-----------------------------------------------------------------
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----------------------------------------------------------------
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data ; Секция инициированных данных
msg: DB 'Введите строку: ',0h ; сообщение
SECTION .bss ; Секция не инициированных данных
buf1: RESB 80 ; Буфер размером 80 байт
SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
mov eax, msg ; запись адреса выводимого сообщения в `EAX`
call sprint ; вызов подпрограммы печати сообщения
mov ecx, buf1 ; запись адреса переменной в `EAX`
mov edx, 80 ; запись длины вводимого сообщения в `EBX`
call sread ; вызов подпрограммы ввода сообщения
call quit ; вызов подпрограммы завершения
```

Рис.3.16. Замена sprintLF на sprint

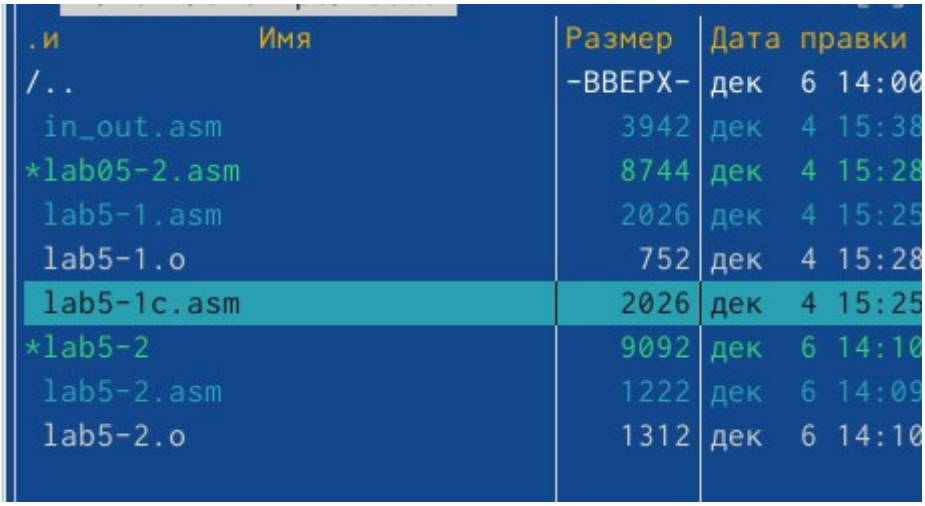
```
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ nasm -f elf lab5-2.asm
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ ld -m elf_i386 -o lab5-2 lab5-2.o
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ ./lab5-2
Введите строку: Тимофеева Милана
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $
```

Рис.3.17. Проверка работы файла

4. Задание для самостоятельной работы

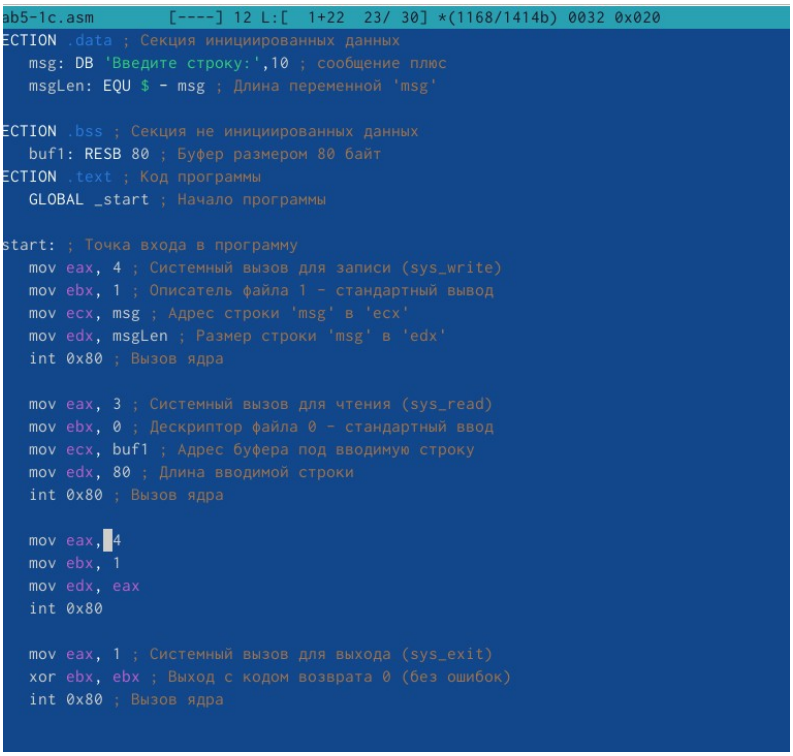
-1. Создала копию файла lab5-1.asm. Внесла изменения в программу (без использования внешнего файла in_out.asm), так чтобы она работала по следующему алгоритму:(рис.4.1.)

- вывести приглашение типа “Введите строку:”;
- ввести строку с клавиатуры;
- вывести введённую строку на экран.



.и	Имя	Размер	Дата правки
/..		-ВВЕРХ-	дек 6 14:00
	in_out.asm	3942	дек 4 15:38
	*lab05-2.asm	8744	дек 4 15:28
	lab5-1.asm	2026	дек 4 15:25
	lab5-1.o	752	дек 4 15:28
	lab5-1c.asm	2026	дек 4 15:25
	*lab5-2	9092	дек 6 14:10
	lab5-2.asm	1222	дек 6 14:09
	lab5-2.o	1312	дек 6 14:10

Рис.4.1. Создание копии файла lab5-1.asm



```
ab5-1c.asm        [----] 12 L:[  1+22  23/ 30] *(1168/1414b) 0032 0x020
ECTION .data ; Секция инициированных данных
   msg: DB 'Введите строку:',10 ; сообщение плюс
   msgLen: EQU $ - msg ; Длина переменной 'msg'

ECTION .bss ; Секция не инициированных данных
   buf1: RESB 80 ; Буфер размером 80 байт
ECTION .text ; Код программы
   GLOBAL _start ; Начало программы

start: ; Точка входа в программу
   mov eax, 4 ; Системный вызов для записи (sys_write)
   mov ebx, 1 ; Описатель файла 1 - стандартный вывод
   mov ecx, msg ; Адрес строки 'msg' в 'ecx'
   mov edx, msgLen ; Размер строки 'msg' в 'edx'
   int 0x80 ; Вызов ядра

   mov eax, 3 ; Системный вызов для чтения (sys_read)
   mov ebx, 0 ; Дескриптор файла 0 - стандартный ввод
   mov ecx, buf1 ; Адрес буфера под вводимую строку
   mov edx, 80 ; Длина вводимой строки
   int 0x80 ; Вызов ядра

   mov eax, 4
   mov ebx, 1
   mov edx, eax
   int 0x80

   mov eax, 1 ; Системный вызов для выхода (sys_exit)
   xor ebx, ebx ; Выход с кодом возврата 0 (без ошибок)
   int 0x80 ; Вызов ядра
```

Рис.4.2. Внесение изменений в программу без использования внешнего файла in_out.asm

-2. Получила исполняемый файл и проверила его работу. На приглашение ввести строку ввел свою фамилию.(рис.4.3.)

```
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ mc

matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ nasm -f elf lab5-1c.asm
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ ld -m elf_i386 -o lab5-1c lab5-1c.o
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ ./lab5-1c
Введите строку:
Тимофеева Милана
Тиматимофеева1@dk7n08 ~/work/arch-pc/lab05 $
```

Рис.4.3. Проверка работы файла

(к сожалению программа выводит только первые две буквы)

-3. Создала копию файла lab5-2.asm. Исправила текст программы с использование подпрограмм из внешнего файла in_out.asm, так чтобы она работала по следующему алгоритму:(рис 4.4.)

- вывести приглашение типа “Введите строку:”;
- ввести строку с клавиатуры;
- вывести введённую строку на экран.

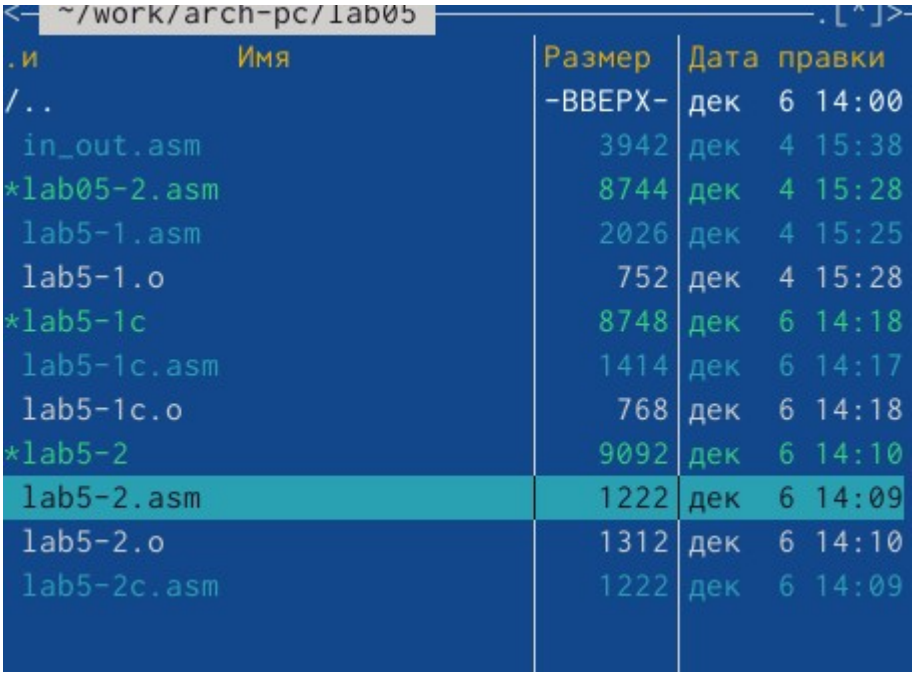


Рис.4.4. Внесение изменений в программу с использованием внешнего файла in_out.asm

-4. Создала исполняемый файл и проверил его работу.(рис.4.5.)

```
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ nasm -f elf lab5-2c.asm
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ ld -m elf_i386 -o lab5-2c lab5-2c.o
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $ ./lab5-2c
Введите строку: Тимофеева Милана
matimofeeva1@dk7n08 ~/work/arch-pc/lab05 $
```

Рис.4.5. Проверка работы файла

5. Вывод

Я приобрела практические навыки работы в Midnight Commander и освоила инструкций языка ассемблера mov и int